

Warehouse Performance Analysis - Part 1

Data Analysis Process

1. Define the Problem

National Warehousing Inc., a U.S.-based supply chain company specializing in order fulfillment and storage, seeks to evaluate the performance of its warehouse branches and analyze customer behavior to reduce costs, improve operational efficiency, and enhance customer satisfaction.

2. Collect & Store the Data

The data for this analysis was already in BigQuery and it is also where the 1st phase of this analysis will be performed. The two tables are **Orders** and **Warehouse** in the dataset, **warehouse_orders**.

Table Dimensions:

Row	order_id	customer_id	warehouse_id	order_date	shipper_date
1	1443	1148	8118	2019-01-24	2019-01-27
2	1444	5073	8118	2019-01-24	2019-01-27
3	1445	2760	8118	2019-01-24	2019-01-27
4	1446	1061	8118	2019-01-24	2019-01-27
5	1447	8350	8118	2019-01-24	2019-01-27
6	1448	4589	8118	2019-01-24	2019-01-27
7	1449	8286	8118	2019-01-24	2019-01-27
8	1450	5124	8118	2019-01-24	2019-01-27
9	1451	1601	8118	2019-01-24	2019-01-27
10	1452	3721	8118	2019-01-24	2019-01-27

Row	warehouse_id	warehouse_alias	maximum_capacity	employee_total	state
1	1543	Somerset Fulfillment Center	210	14	KY
2	2270	Bowling Green Warehouse	280	13	KY
3	9080	Frankfort Fulfillment Center	235	5	KY
4	2666	Lansing Fulfillment Center	290	16	MI
5	3961	Lansing Storage Warehouse	740	22	MI
6	8118	Ann Arbor Fulfillment Center	780	17	MI
7	3417	Gatlinburg Warehouse	620	6	TN
8	4338	Knoxville Fulfillment Center	215	13	TN
9	6509	Memphis Fulfillment Center	755	22	TN
10	9831	Clarsvill Warehouse	400	16	TN

Orders - 10,000 x 5

Warehouse - 10 x 5

There will also be 7 additional tables from queries below that will be used in Power BI to visualize the results.

3. Clean & Prepare the Data

The table, **Orders**, appeared clean at first glance but when I sorted it by the **order_date** column, I spotted an outlier with a date of 2023-09-01.

```
1 SELECT order_date
2 FROM `my-data-project-373523.warehouse_orders.Orders`
3 ORDER BY order_date DESC; |
```

Row	order_date
1	2023-09-01
2	2019-12-30
3	2019-12-30
4	2019-12-30
5	2019-12-30
6	2019-12-30
7	2019-12-30
8	2019-12-30
9	2019-12-30
10	2019-12-30
11	2019-12-30
12	2019-12-30
13	2019-12-30
14	2019-12-30
15	2019-12-30

So, to check if this was a valid record or not, I checked to see when was the table last updated and the date read 2-27-2023.

Orders

Query

Open in

Share

Copy

Schema

Details

Preview

Table Explorer

Preview

Ins

Table info

Table ID

my-data-project-373523.warehouse_orders.Orders

Created

Feb 23, 2023, 7:51:52 PM UTC-5

Last modified

Feb 27, 2023, 8:13:11 PM UTC-5

Table expiration

NEVER

Data location

US

Default collation

Default rounding mode

ROUNDING_MODE_UNSPECIFIED

Case insensitive

false

Description

Labels

Primary key(s)

Tags

Storage info

Number of rows

10,000

To handle this one row, I simply excluded it using the `WHERE` clause in all of the queries.

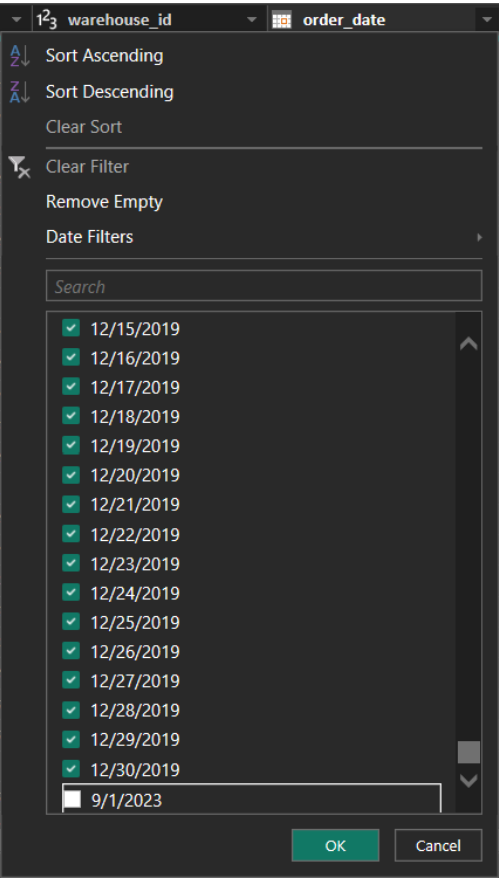
```
1 SELECT
2   FORMAT('%d', COUNT(DISTINCT order_id)) AS `total orders`,
3   FORMAT('%d', COUNT(DISTINCT customer_id)) AS `total customers`
4 FROM `my-data-project-373523.warehouse_orders.Orders`
5 WHERE order_date != '2023-09-01';
```

Query completed

Query results

Job information Results Chart JSON Execution details			
Row	total orders	total customers	
1	9,999	6,065	

In Power BI:
I filtered the row out of the data set using the Power Query Editor.



I also had to remove the comma from the **number of orders** column in the **warehouse-total-orders** table to transform the value from string to number.

warehouse_orders_Schema[Name="warehouse-total-orders",Kind="Table"]			
warehouse	state	number of orders	
1 Somerset Fulfillment Center	KY	548	
2 Frankfort Fulfillment Center	KY	500	
3 Lansing Fulfillment Center	MI	3,178	
4 Ann Arbor Fulfillment Center	MI	3,027	
5 Memphis Fulfillment Center	TN	2,483	
6 Knoxville Fulfillment Center	TN	343	

Replace Values

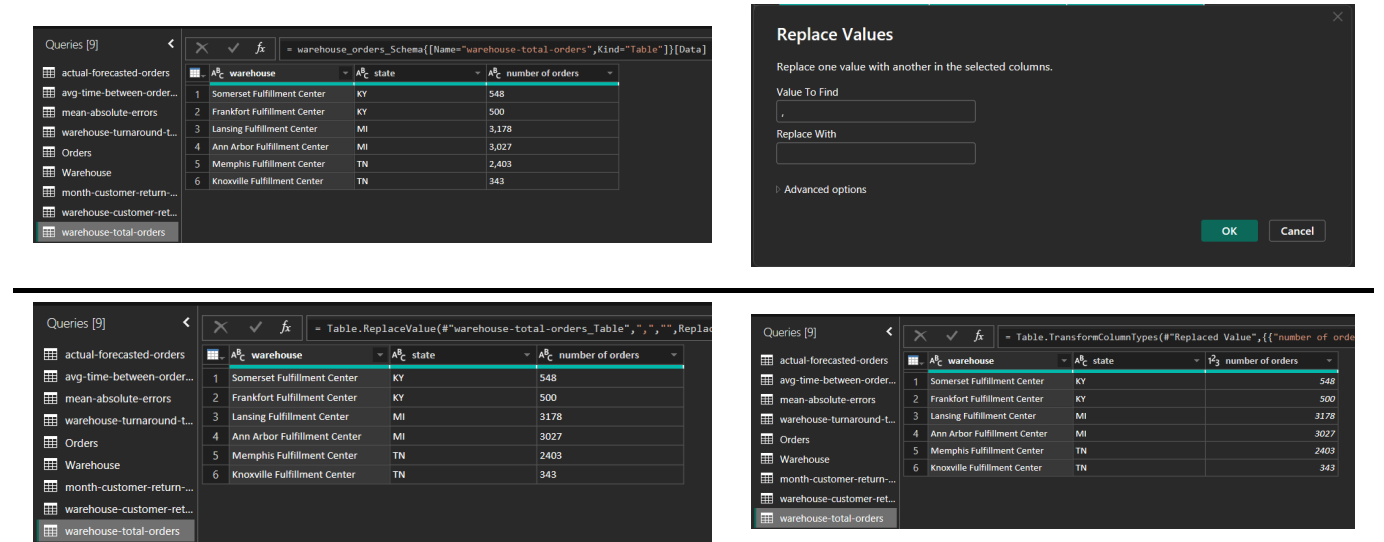
Replace one value with another in the selected columns.

Value To Find

Replace With

Advanced options

OK Cancel



4. Analyze the Data

What are the total orders and customers?

```
1 SELECT
2   FORMAT('%d', COUNT(DISTINCT order_id)) AS 'total orders',
3   FORMAT('%d', COUNT(DISTINCT customer_id)) AS 'total customers'
4 FROM 'my-data-project-373523.warehouse_orders.Orders'
5 WHERE order_date != '2023-09-01';
```

Query completed

Query results

Job information	Results	Chart	JSON	Execution details
Row	total orders	total customers		
1	9,999	6,065		

What's the amount of shipments per warehouse?

```
1 SELECT
2   w.warehouse_alias AS warehouse,
3   w.state,
4   FORMAT('%d', COUNT(o.order_id)) AS 'total orders'
5 FROM 'warehouse_orders.Orders' o
6 RIGHT JOIN 'warehouse_orders.Warehouse' w ON o.warehouse_id = w.warehouse_id
7 WHERE o.order_date IS NULL OR o.order_date != '2023-09-01'
8 GROUP BY warehouse, w.state
9 ORDER BY COUNT(o.order_id) DESC, warehouse;
```

Query completed

Query results

Job information	Results	Chart	JSON	Execution details	Execution gra
Row	warehouse	state	total orders		
1	Lansing Fulfillment Center	MI	3,178		
2	Ann Arbor Fulfillment Center	MI	3,027		
3	Memphis Fulfillment Center	TN	2,403		
4	Somerset Fulfillment Center	KY	548		
5	Frankfort Fulfillment Center	KY	500		
6	Knoxville Fulfillment Center	TN	343		
7	Bowling Green Warehouse	KY	0		
8	Clarsvill Warehouse	TN	0		
9	Gatlinburg Warehouse	TN	0		
10	Lansing Storage Warehouse	MI	0		

What's the capacity being wasted at the warehouses with no shipments?

1SELECT

2w.warehouse_alias AS warehouse,

3w.employee_total AS 'total employees',

4w.state,

5FORMAT('%d', CAST(w.maximum_capacity * 55.44 AS INT64)) AS 'max capacity cu ft'

6FROM `warehouse\_orders.Orders` o

7RIGHT JOIN `warehouse\_orders.Warehouse` w ON o.warehouse_id = w.warehouse_id

8GROUP BY warehouse, 'total employees', w.state, 'max capacity cu ft'

9HAVING COUNT(o.order_id) = 0

10ORDER BY 'max capacity cu ft' DESC;

Query completed

Query results

Job informationResultsChartJSONExecution detailsExecution graph

Row	warehouse	total employees	state	max capacity cu ft
1	Lansing Storage Warehouse	22	MI	41,026
2	Gatlinburg Warehouse	6	TN	34,373
3	Clarsvill Warehouse	16	TN	22,176
4	Bowling Green Warehouse	13	KY	15,523

How many orders are these warehouses getting each month?

1SELECT

2FORMAT_DATE('%B', o.order_date) AS month,

3w.warehouse_alias AS warehouse,

4COUNT(o.order_id) AS 'number of orders'

5FROM `warehouse\_orders.Orders` o

6LEFT JOIN `warehouse\_orders.Warehouse` w ON o.warehouse_id = w.warehouse_id

7WHERE o.order_date != '2023-09-01'

8GROUP BY month, EXTRACT(MONTH FROM o.order_date), warehouse

9ORDER BY EXTRACT(MONTH FROM o.order_date), 'number of orders' DESC;

Row	month	warehouse	number of orders
1	January	Ann Arbor Fulfillment Center	267
2	January	Lansing Fulfillment Center	265
3	January	Memphis Fulfillment Center	219
4	January	Somerset Fulfillment Center	45
5	January	Frankfort Fulfillment Center	33
6	January	Knoxville Fulfillment Center	30
7	February	Lansing Fulfillment Center	257
8	February	Ann Arbor Fulfillment Center	243
9	February	Memphis Fulfillment Center	181
10	February	Somerset Fulfillment Center	60
11	February	Frankfort Fulfillment Center	37
12	February	Knoxville Fulfillment Center	27
13	March	Ann Arbor Fulfillment Center	260
14	March	Lansing Fulfillment Center	260
15	March	Memphis Fulfillment Center	195

Row	month	warehouse	number of orders
58	October	Somerset Fulfillment Center	48
59	October	Frankfort Fulfillment Center	42
60	October	Knoxville Fulfillment Center	39
61	November	Ann Arbor Fulfillment Center	251
62	November	Lansing Fulfillment Center	245
63	November	Memphis Fulfillment Center	196
64	November	Somerset Fulfillment Center	47
65	November	Frankfort Fulfillment Center	39
66	November	Knoxville Fulfillment Center	26
67	December	Lansing Fulfillment Center	269
68	December	Ann Arbor Fulfillment Center	246
69	December	Memphis Fulfillment Center	195
70	December	Frankfort Fulfillment Center	50
71	December	Somerset Fulfillment Center	37
72	December	Knoxville Fulfillment Center	29

How fast are these orders being processed?

```
1 SELECT
2   DATE_TRUNC(o.order_date, WEEK(MONDAY)) AS 'order week',
3   w.warehouse_alias AS warehouse,
4   AVG(DATE_DIFF(o.shipper_date, o.order_date, DAY)) AS 'average weekly turnaround'
5 FROM   'warehouse_orders.Orders' o
6 LEFT JOIN 'warehouse_orders.Warehouse' w ON o.warehouse_id = w.warehouse_id
7 WHERE  o.order_date != '2023-09-01'
8 GROUP BY 'order week', warehouse
9 ORDER BY 'order week';
```

Query completed

Query results

Job information	Results	Chart	JSON	Execution details	Execution
Row	order week	warehouse	average weekly turnaround		
1	2018-12-31	Ann Arbor Fulfillment Center	3.0		
2	2018-12-31	Lansing Fulfillment Center	3.0		
3	2018-12-31	Frankfort Fulfillment Center	3.0		
4	2018-12-31	Somerset Fulfillment Center	3.0		
5	2018-12-31	Memphis Fulfillment Center	3.0		
6	2018-12-31	Knoxville Fulfillment Center	3.0		
7	2019-01-07	Ann Arbor Fulfillment Center	3.0		
8	2019-01-07	Lansing Fulfillment Center	3.0		
9	2019-01-07	Frankfort Fulfillment Center	3.0		
10	2019-01-07	Somerset Fulfillment Center	3.0		
11	2019-01-07	Memphis Fulfillment Center	3.0		
12	2019-01-07	Knoxville Fulfillment Center	3.0		
13	2019-01-14	Ann Arbor Fulfillment Center	3.0		

```
1 SELECT
2   DATE_TRUNC(o.order_date, WEEK(MONDAY)) AS 'order week',
3   w.warehouse_alias AS warehouse,
4   AVG(DATE_DIFF(o.shipper_date, o.order_date, DAY)) AS 'average weekly turnaround'
5 FROM   'warehouse_orders.Orders' o
6 LEFT JOIN 'warehouse_orders.Warehouse' w ON o.warehouse_id = w.warehouse_id
7 WHERE  o.order_date != '2023-09-01'
8 GROUP BY 'order week', warehouse
9 ORDER BY 'order week';
```

Query completed

Query results

Job information	Results	Chart	JSON	Execution details	Execution
Row	order week	warehouse	average weekly turnaround		
301	2019-12-16	Ann Arbor Fulfillment Center	3.0		
302	2019-12-16	Lansing Fulfillment Center	3.0		
303	2019-12-16	Frankfort Fulfillment Center	3.0		
304	2019-12-16	Somerset Fulfillment Center	3.0		
305	2019-12-16	Memphis Fulfillment Center	3.0		
306	2019-12-16	Knoxville Fulfillment Center	3.0		
307	2019-12-23	Ann Arbor Fulfillment Center	3.0		
308	2019-12-23	Lansing Fulfillment Center	3.0		
309	2019-12-23	Frankfort Fulfillment Center	3.0		
310	2019-12-23	Somerset Fulfillment Center	3.0		
311	2019-12-23	Memphis Fulfillment Center	3.0		
312	2019-12-23	Knoxville Fulfillment Center	3.0		
313	2019-12-30	Ann Arbor Fulfillment Center	3.0		

Rank warehouses based off of the average number of orders per week & month

```
1 WITH weeklyOrders AS (
2   SELECT
3     DATE_TRUNC(o.order_date, WEEK(MONDAY)) AS 'order week',
4     w.warehouse_alias AS warehouse,
5     COUNT(o.order_id) AS 'number of orders'
6 FROM   'warehouse_orders.Orders' o
7 LEFT JOIN 'warehouse_orders.Warehouse' w ON o.warehouse_id = w.warehouse_id
8 WHERE  o.order_date != '2023-09-01'
9 GROUP BY 'order week', warehouse
10 ),
11
12 avgOrders AS (
13   SELECT
14     warehouse,
15     ROUND(AVG('number of orders'),2) AS 'average orders per week',
16     RANK() OVER(ORDER BY AVG('number of orders') DESC) AS 'rank by average orders'
17 FROM   weeklyOrders
18 GROUP BY warehouse
19 )
20
21 SELECT
22   warehouse,
23   'average orders per week'
24 FROM   avgOrders
25 ORDER BY 'rank by average orders';
```

Query completed

Query results

Job information	Results	Chart	JSON	Execution details	Execution g
Row	warehouse	average orders per week			
1	Lansing Fulfillment Center	59.96			
2	Ann Arbor Fulfillment Center	57.11			
3	Memphis Fulfillment Center	45.34			
4	Somerset Fulfillment Center	10.54			
5	Frankfort Fulfillment Center	9.43			
6	Knoxville Fulfillment Center	6.47			

```
1 WITH monthlyOrders AS (
2   SELECT
3     w.warehouse_alias AS warehouse,
4     EXTRACT(MONTH FROM o.order_date) AS 'order month',
5     COUNT(o.order_id) AS 'number of orders'
6 FROM   'warehouse_orders.Orders' o
7 LEFT JOIN 'warehouse_orders.Warehouse' w ON o.warehouse_id = w.warehouse_id
8 WHERE  o.order_date != '2023-09-01'
9 GROUP BY 'order month', warehouse
10 ),
11
12 avgOrders AS (
13   SELECT
14     warehouse,
15     ROUND(AVG('number of orders'),2) AS 'average orders per month',
16     RANK() OVER(ORDER BY AVG('number of orders') DESC) AS 'rank by average orders'
17 FROM   monthlyOrders
18 GROUP BY warehouse
19 )
20
21 SELECT
22   warehouse,
23   'average orders per month'
24 FROM   avgOrders
25 ORDER BY 'rank by average orders' ASC;
```

Query completed

Query results

Job information	Results	Chart	JSON	Execution details	Execution g
Row	warehouse	average orders per month			
1	Lansing Fulfillment Center	264.83			
2	Ann Arbor Fulfillment Center	252.25			
3	Memphis Fulfillment Center	200.25			
4	Somerset Fulfillment Center	45.67			
5	Frankfort Fulfillment Center	41.67			
6	Knoxville Fulfillment Center	28.58			

Get a 5-day forecast for orders (Monday-Friday); edited the query so that the first week did not have NULL values

```
1 WITH weeklyOrders AS (  
2   SELECT  
3     DATE_TRUNC(o.order_date, WEEK(MONDAY)) AS 'order week',  
4     w.warehouse_alias AS warehouse,  
5     COUNT(o.order_id) AS 'number of orders'  
6   FROM 'warehouse_orders.Orders' o  
7   LEFT JOIN 'warehouse_orders.Warehouse' w ON o.warehouse_id = w.warehouse_id  
8   WHERE o.order_date != '2023-09-01'  
9   GROUP BY 'order week', warehouse  
10 )  
11  
12 SELECT  
13   'order week',  
14   warehouse,  
15   AVG('number of orders') OVER(PARTITION BY warehouse ORDER BY 'order week' ROWS BETWEEN 4 PRECEDING AND 1 PRECEDING) AS 'forecasted orders'  
16 FROM weeklyOrders  
17 ORDER BY 'order week';
```

Query completed
Processing location: US

Job information	Results	Chart	JSON	Execution details	Execution graph
Row	order week	warehouse	forecasted orders		
1	2018-12-31	Somerset Fulfillment Center	null		
2	2018-12-31	Memphis Fulfillment Center	null		
3	2018-12-31	Knoxville Fulfillment Center	null		
4	2018-12-31	Frankfort Fulfillment Center	null		
5	2018-12-31	Lansing Fulfillment Center	null		
6	2018-12-31	Ann Arbor Fulfillment Center	null		
7	2019-01-07	Somerset Fulfillment Center	9.0		
8	2019-01-07	Memphis Fulfillment Center	45.0		
9	2019-01-07	Knoxville Fulfillment Center	10.0		
10	2019-01-07	Frankfort Fulfillment Center	8.0		
11	2019-01-07	Lansing Fulfillment Center	55.0		
12	2019-01-07	Ann Arbor Fulfillment Center	57.0		

```
1 WITH weeklyOrders AS (  
2   SELECT  
3     DATE_TRUNC(o.order_date, WEEK(MONDAY)) AS 'order week',  
4     w.warehouse_alias AS warehouse,  
5     COUNT(o.order_id) AS 'number of orders'  
6   FROM 'warehouse_orders.Orders' o  
7   LEFT JOIN 'warehouse_orders.Warehouse' w ON o.warehouse_id = w.warehouse_id  
8   WHERE o.order_date != '2023-09-01'  
9   GROUP BY 'order week', warehouse  
10 )  
11  
12 forecasted AS (  
13   SELECT  
14     'order week',  
15     warehouse,  
16     AVG('number of orders') OVER(PARTITION BY warehouse ORDER BY 'order week' ROWS BETWEEN 4 PRECEDING AND 1 PRECEDING) AS 'forecasted orders'  
17 FROM weeklyOrders  
18 )  
19  
20 SELECT  
21   'order week',  
22   warehouse,  
23   TRUNC('forecasted orders',2) AS 'forecasted orders'  
24 FROM forecasted  
25 WHERE 'forecasted orders' IS NOT NULL  
26 ORDER BY 'order week';
```

Query completed
Processing location: US

Job information	Results	Chart	JSON	Execution details	Execution graph
Row	order week	warehouse	forecasted orders		
1	2019-01-07	Lansing Fulfillment Center	55.0		
2	2019-01-07	Knoxville Fulfillment Center	10.0		
3	2019-01-07	Somerset Fulfillment Center	9.0		
4	2019-01-07	Frankfort Fulfillment Center	8.0		
5	2019-01-07	Memphis Fulfillment Center	45.0		
6	2019-01-07	Ann Arbor Fulfillment Center	57.0		
7	2019-01-14	Lansing Fulfillment Center	58.0		
8	2019-01-14	Knoxville Fulfillment Center	9.0		
9	2019-01-14	Somerset Fulfillment Center	9.0		
10	2019-01-14	Frankfort Fulfillment Center	7.5		
11	2019-01-14	Memphis Fulfillment Center	52.0		
12	2019-01-14	Ann Arbor Fulfillment Center	58.0		
13	2019-01-21	Lansing Fulfillment Center	55.33		
14	2019-01-21	Knoxville Fulfillment Center	8.33		

Compare actual amount of orders to 5-day forecasted amount of orders

```
1 WITH weeklyOrders AS (  
2   SELECT  
3     DATE_TRUNC(o.order_date, WEEK(MONDAY)) AS 'order week',  
4     w.warehouse_alias AS warehouse,  
5     COUNT(o.order_id) AS 'number of orders'  
6   FROM 'warehouse_orders.Orders' o  
7   LEFT JOIN 'warehouse_orders.Warehouse' w ON o.warehouse_id = w.warehouse_id  
8   WHERE o.order_date != '2023-09-01'  
9   GROUP BY 'order week', warehouse  
10 ),  
11  
12 forecasted AS (  
13   SELECT  
14     'order week',  
15     warehouse,  
16     'number of orders' AS 'actual orders',  
17     AVG('number of orders') OVER(PARTITION BY warehouse ORDER BY 'order week' ROWS BETWEEN 4 PRECEDING AND 1 PRECEDING) AS 'forecasted orders'  
18 FROM weeklyOrders  
19 )  
20  
21 SELECT  
22   'order week',  
23   warehouse,  
24   'actual orders',  
25   TRUNC('forecasted orders',2) AS 'forecasted orders'  
26 FROM forecasted  
27 WHERE 'forecasted orders' IS NOT NULL  
28 ORDER BY 'order week';
```

Query completed
Processing location: US

Job information	Results	Chart	JSON	Execution details	Execution graph
Row	order week	warehouse	actual orders	forecasted orders	
1	2019-01-07	Lansing Fulfillment Center	61	55.0	
2	2019-01-07	Somerset Fulfillment Center	9	9.0	
3	2019-01-07	Frankfort Fulfillment Center	7	8.0	
4	2019-01-07	Memphis Fulfillment Center	59	45.0	
5	2019-01-07	Knoxville Fulfillment Center	8	10.0	
6	2019-01-07	Ann Arbor Fulfillment Center	59	57.0	
7	2019-01-14	Lansing Fulfillment Center	50	58.0	
8	2019-01-14	Somerset Fulfillment Center	13	9.0	
9	2019-01-14	Frankfort Fulfillment Center	7	7.5	
10	2019-01-14	Memphis Fulfillment Center	47	52.0	
11	2019-01-14	Knoxville Fulfillment Center	7	9.0	
12	2019-01-14	Ann Arbor Fulfillment Center	62	58.0	

Which warehouses have orders that either meet or are close to the amount of forecasted orders (have the smallest mean absolute error)? Note, there is the MAE before and after I filtered out the order with a date of 2023-09-01

```
1 WITH weeklyOrders AS (  
2   SELECT  
3     DATE_TRUNC(o.order_date, WEEK(MONDAY)) AS 'order week',  
4     w.warehouse_alias AS warehouse,  
5     COUNT(o.order_id) AS 'number of orders'  
6   FROM warehouse_orders.Orders o  
7   LEFT JOIN warehouse_orders.Warehouse w ON o.warehouse_id = w.warehouse_id  
8   WHERE o.order_date != '2023-09-01'  
9   GROUP BY 'order week', warehouse  
10 ),  
11  
12 forecasted AS (  
13   SELECT  
14     'order week',  
15     warehouse,  
16     'number of orders' AS 'actual orders',  
17     TRUNC(AVG('number of orders') OVER(PARTITION BY warehouse ORDER BY 'order week' ROWS BETWEEN 4 PRECEDING AND 1 PRECEDING),2) AS 'forecasted orders'  
18   FROM weeklyOrders  
19 )  
20  
21 SELECT  
22   warehouse,  
23   TRUNC(AVG(ABS('actual orders' - 'forecasted orders')),2) AS 'mean absolute error'  
24 FROM forecasted  
25 WHERE 'forecasted orders' IS NOT NULL  
26 GROUP BY warehouse  
27 ORDER BY 'mean absolute error';
```

Query completed

Processing location: US

Query results

Job information Results Chart JSON Execution details Execution graph

Row	warehouse	mean absolute error
1	Somerset Fulfillment Center	2.43
2	Knoxville Fulfillment Center	2.46
3	Frankfort Fulfillment Center	2.6
4	Lansing Fulfillment Center	7.51
5	Memphis Fulfillment Center	8.37
6	Ann Arbor Fulfillment Center	8.69

Row	warehouse	mean absolute error
1	Somerset Fulfillment Center	2.43
2	Knoxville Fulfillment Center	2.46
3	Frankfort Fulfillment Center	2.6
4	Lansing Fulfillment Center	7.51
5	Memphis Fulfillment Center	8.37
6	Ann Arbor Fulfillment Center	9.29

Row	warehouse	mean absolute error
1	Somerset Fulfillment Center	2.43
2	Knoxville Fulfillment Center	2.46
3	Frankfort Fulfillment Center	2.6
4	Lansing Fulfillment Center	7.51
5	Memphis Fulfillment Center	8.37
6	Ann Arbor Fulfillment Center	8.69

There's a .6 difference in the MAE for the Ann Arbor facility just because of that one row.

What is the percentage of returning customers (retention rate)?

```
1 WITH customerOrders AS (  
2   SELECT  
3     customer_id,  
4     COUNT(order_id) AS 'total orders'  
5   FROM warehouse_orders.Orders  
6   WHERE order_date != '2023-09-01'  
7   GROUP BY customer_id  
8 )  
9  
10 SELECT ROUND(COUNTIF('total orders' > 1) / COUNT(*) * 100, 2) AS 'retention rate'  
11 FROM customerOrders;
```

Query completed

Query results

Job information Results Chart JSON Execution details Execution graph

Row	retention rate
1	44.86

How Frequently Do Individual Customers Reorder (Repeat Purchase Cycle)?


```
1 WITH orderGaps AS (  
2   SELECT  
3     customer_id,  
4     order_date,  
5     LEAD(order_date) OVER(PARTITION BY customer_id ORDER BY order_date) AS 'next order date'  
6   FROM warehouse_orders.Orders  
7   WHERE order_date != '2023-09-01'  
8 ),  
9 totalOrders AS (  
10  SELECT  
11    customer_id,  
12    COUNT(order_id) AS 'total orders'  
13  FROM warehouse_orders.Orders  
14  GROUP BY customer_id  
15 )  
16  
17 SELECT  
18   a.customer_id,  
19   b.'total orders',  
20   ROUND(AVG(DATE_DIFF('next order date', a.order_date, DAY)),2) AS 'avg days between orders'  
21 FROM orderGaps a  
22 JOIN totalOrders b ON a.customer_id = b.customer_id  
23 WHERE b.'total orders' > 1 AND 'next order date' IS NOT NULL  
24 GROUP BY a.customer_id, b.'total orders'  
25 ORDER BY b.'total orders', 'avg days between orders', a.customer_id;
```

Query completed

Processing location: US

Query results

Job information	Results	Chart	JSON	Execution details	Execution graph
Row	customer_id	total orders	avg days between orders		
1	2005	2	0.0		
2	4896	2	0.0		
3	7059	2	0.0		
4	7251	2	0.0		
5	2839	2	1.0		
6	2824	2	1.0		
7	3846	2	1.0		
8	4079	2	1.0		
9	4127	2	1.0		
10	6059	2	1.0		
11	6488	2	1.0		
12	8287	2	1.0		
13	8715	2	1.0		
14	8800	2	1.0		

Row	customer_id	total orders	avg days between orders
1351	6862	2	178.0
1352	7370	2	178.0
1353	8031	2	178.0
1354	8641	2	178.0
1355	2163	2	179.0
1356	4650	2	179.0
1357	1960	2	180.0
1358	1968	2	180.0
1359	4215	2	180.0
1360	7574	2	180.0
1361	8156	2	180.0
1362	9515	2	180.0
1363	9684	2	180.0
1364	9871	2	180.0
1365	2738	2	181.0
1366	7081	2	181.0
1367	7242	2	181.0
1368	7664	2	181.0
1369	8868	2	181.0
1370	1153	2	182.0
1371	2625	2	182.0
1372	3179	2	182.0
1373	5670	2	182.0
1374	5723	2	182.0
1375	6331	2	182.0
1376	6882	2	182.0
1377	7846	2	182.0
1378	1395	2	183.0
1379	4006	2	183.0
1380	4028	2	183.0

Row	customer_id	total orders	avg days between orders
2701	8897	5	79.0
2702	5528	5	82.25
2703	5636	5	82.25
2704	1646	5	82.5
2705	6689	5	83.75
2706	3423	5	84.5
2707	3684	6	31.0
2708	8105	6	39.2
2709	2864	6	41.6
2710	2748	6	43.6
2711	6627	6	44.6
2712	4524	6	47.4
2713	2387	6	50.6
2714	1601	6	53.4
2715	5816	6	59.0
2716	6407	6	63.0
2717	9118	6	64.6
2718	9682	6	68.0
2719	2580	6	69.0
2720	3548	7	34.67
2721	6633	7	60.17

There are 2,721 returning customers; Average number of days between orders range from 0.0 - 348.0;
Customers with more orders have less days between orders

Get the customer return frequency over the year

Row	month	month name	returning customers	non-returning customers
1	1	January	48	763
2	2	February	104	676
3	3	March	166	647
4	4	April	238	536
5	5	May	244	515
6	6	June	301	456
7	7	July	352	451
8	8	August	409	425
9	9	September	442	358
10	10	October	487	380
11	11	November	454	308
12	12	December	494	296

Get the customer return frequency for each warehouse

```
1 SELECT
2   warehouse_id,
3   warehouse_alias AS 'warehouse',
4   COUNT(DISTINCT IF(order_rank > 1, customer_id, NULL)) AS 'returning customers',
5   COUNT(DISTINCT customer_id) - COUNT(DISTINCT IF(order_rank > 1, customer_id, NULL)) AS 'non-returning customers'
6 FROM (
7   SELECT
8     o.customer_id,
9     w.warehouse_id,
10    w.warehouse_alias,
11    ROW_NUMBER() OVER(PARTITION BY o.customer_id, o.warehouse_id ORDER BY o.order_date) AS order_rank
12  FROM warehouse_orders.Orders o
13  LEFT JOIN warehouse_orders.Warehouse w ON o.warehouse_id = w.warehouse_id
14  WHERE o.order_date != '2023-09-01'
15 )
16 GROUP BY warehouse_id, 'warehouse'
17 ORDER BY 'returning customers' DESC;
```

✔ Query completed

Processing location: US ✕

Query results

Job information						Results	Chart	JSON	Execution details	Execution graph
Row	warehouse_id	warehouse	returning customers	non-returning customers						
1	2666	Lansing Fulfillment Center	449	2231						
2	8118	Ann Arbor Fulfillment Center	407	2158						
3	6509	Memphis Fulfillment Center	259	1857						
4	9080	Frankfort Fulfillment Center	15	470						
5	1543	Somerset Fulfillment Center	13	521						
6	4338	Knoxville Fulfillment Center	7	329						

Steps 5 & 6 for this analysis will continue [here](#)